

Client Side Environment

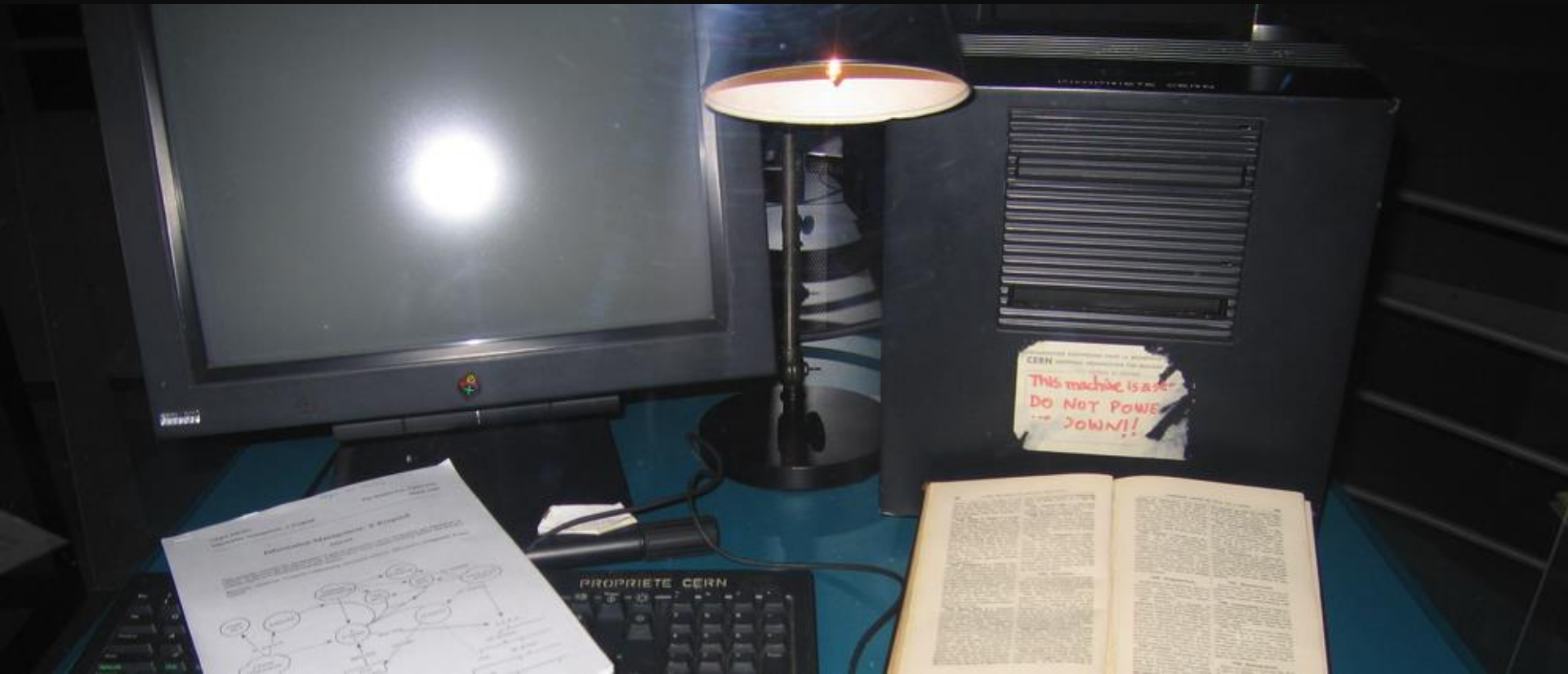
Internet Technology
ITECH

The Creator

(of the [web](#))



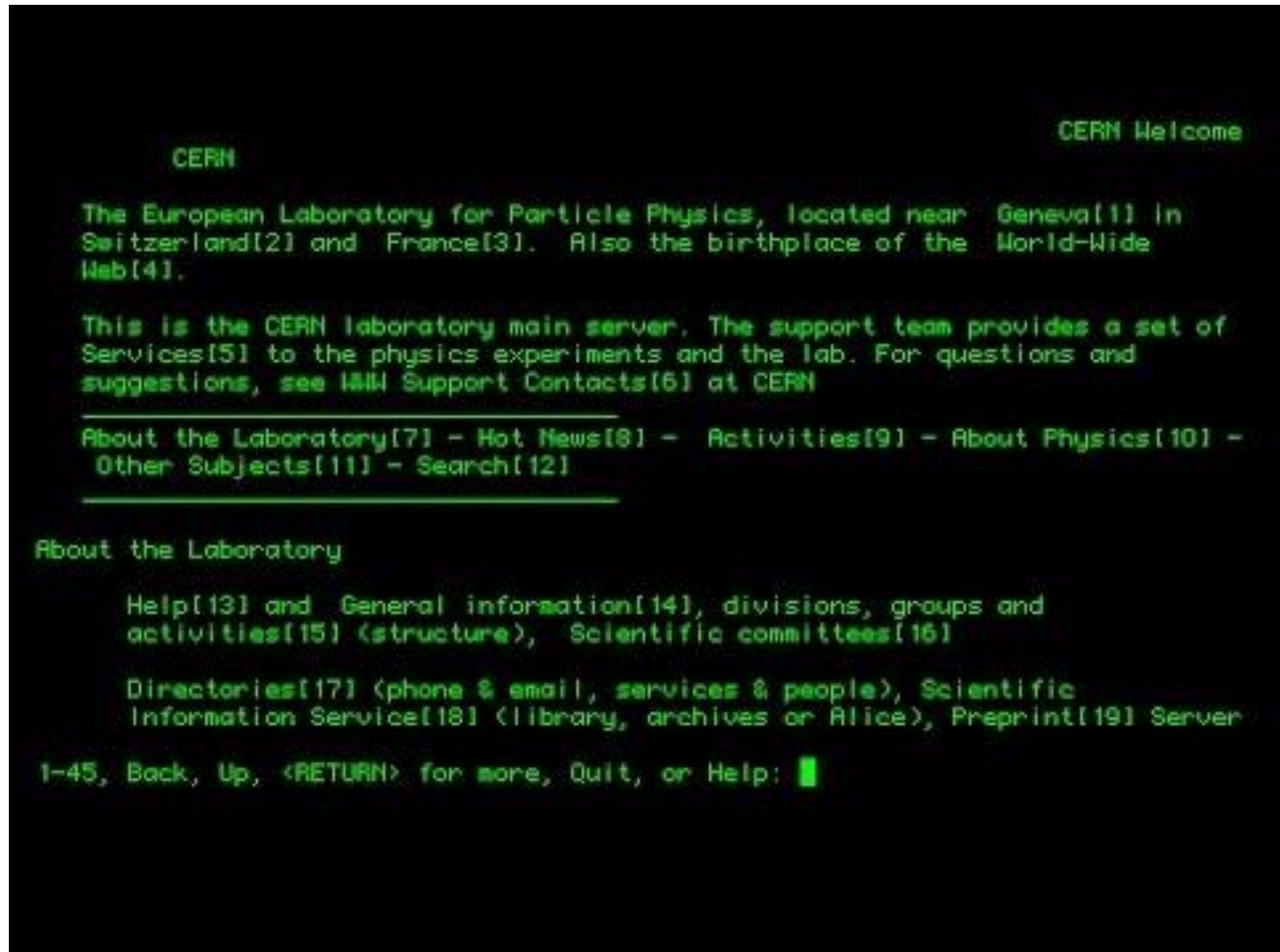
The Whole Stack



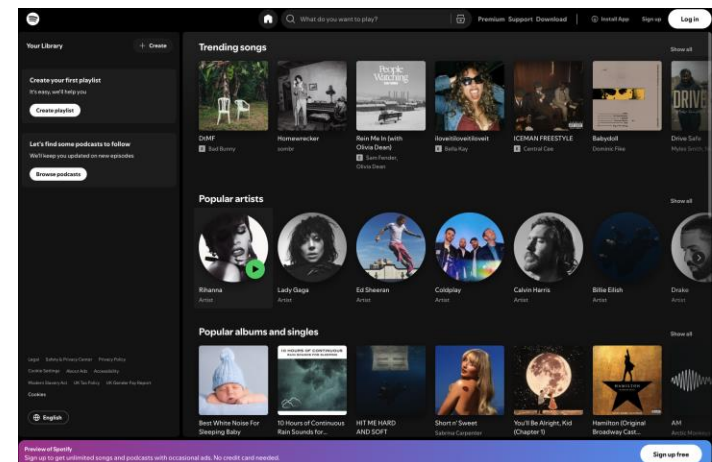
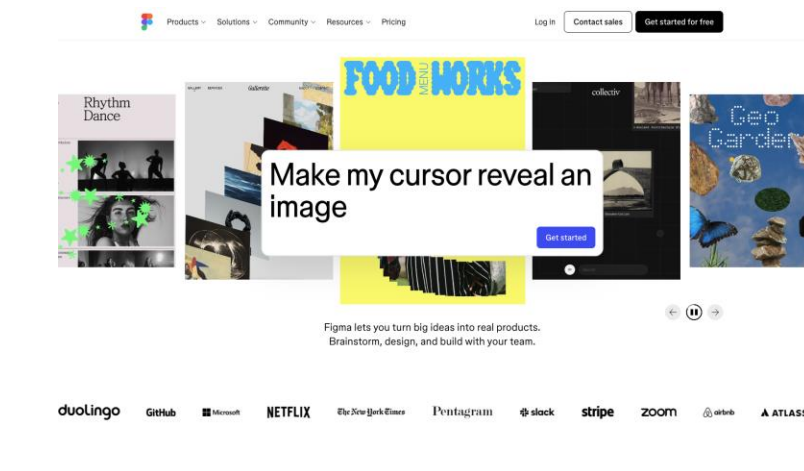
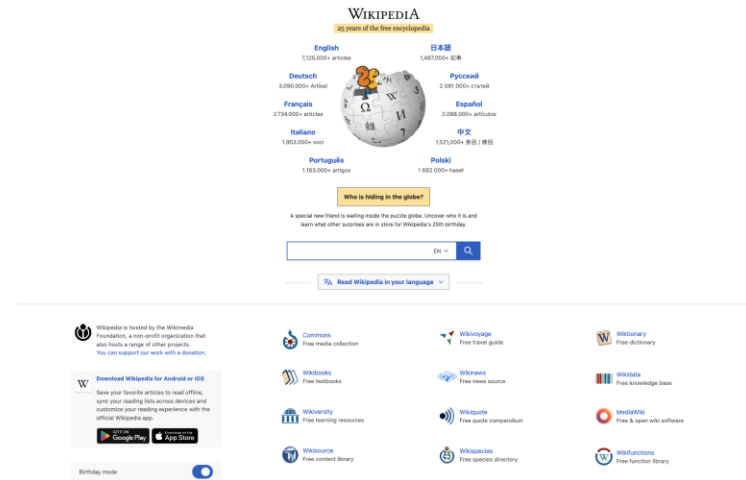
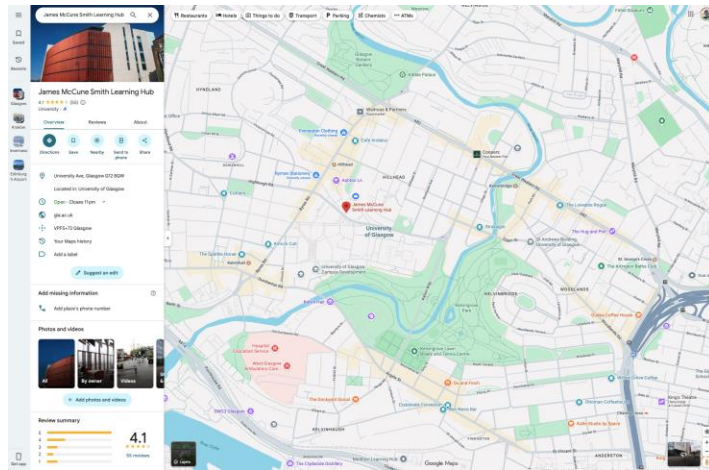
- HTTP
- HTTP Server
- HTML
- Web Browser / Editor / Usenet / FTP
- only ran on NeXT

Mostly developed and in place by
1990

The first web page



Web pages in 2026



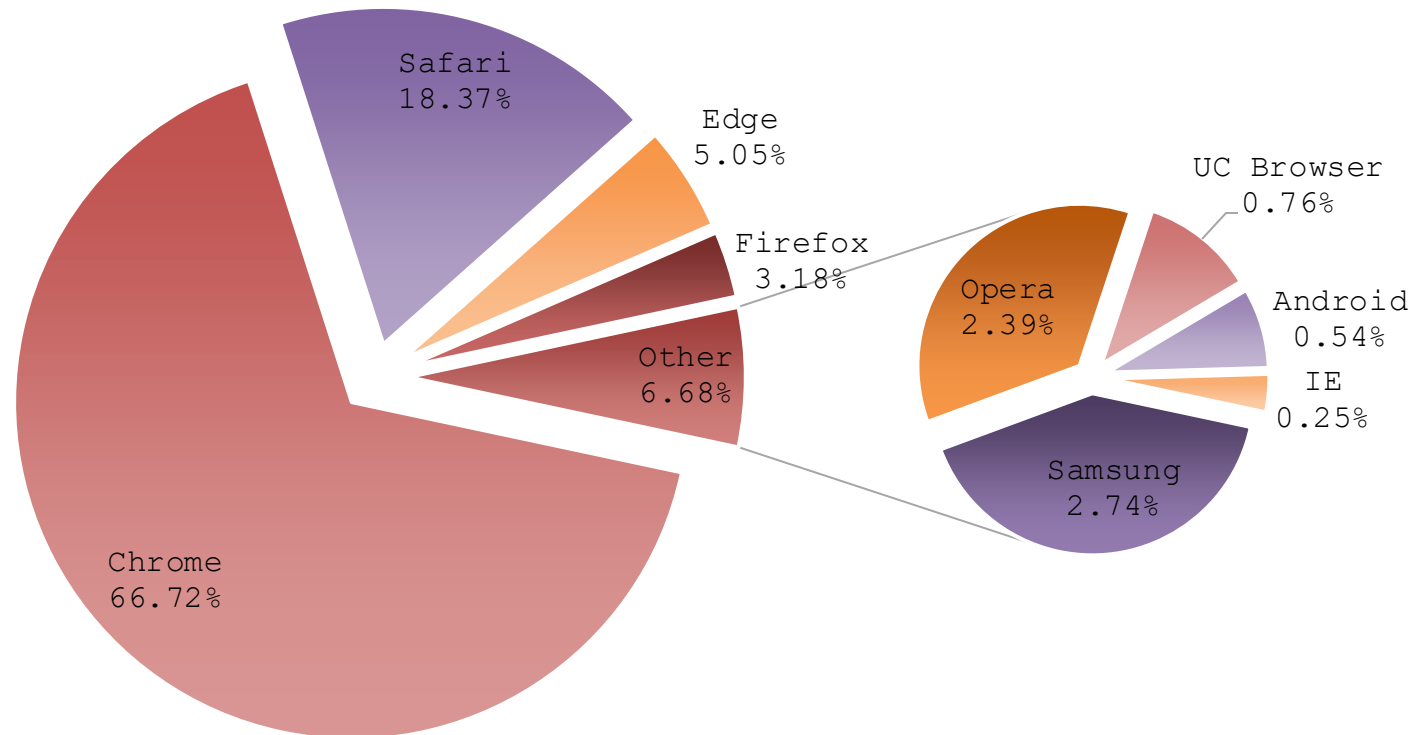
- What has drastically changed? **The web browser!**



Web
Browser

Browsers Now

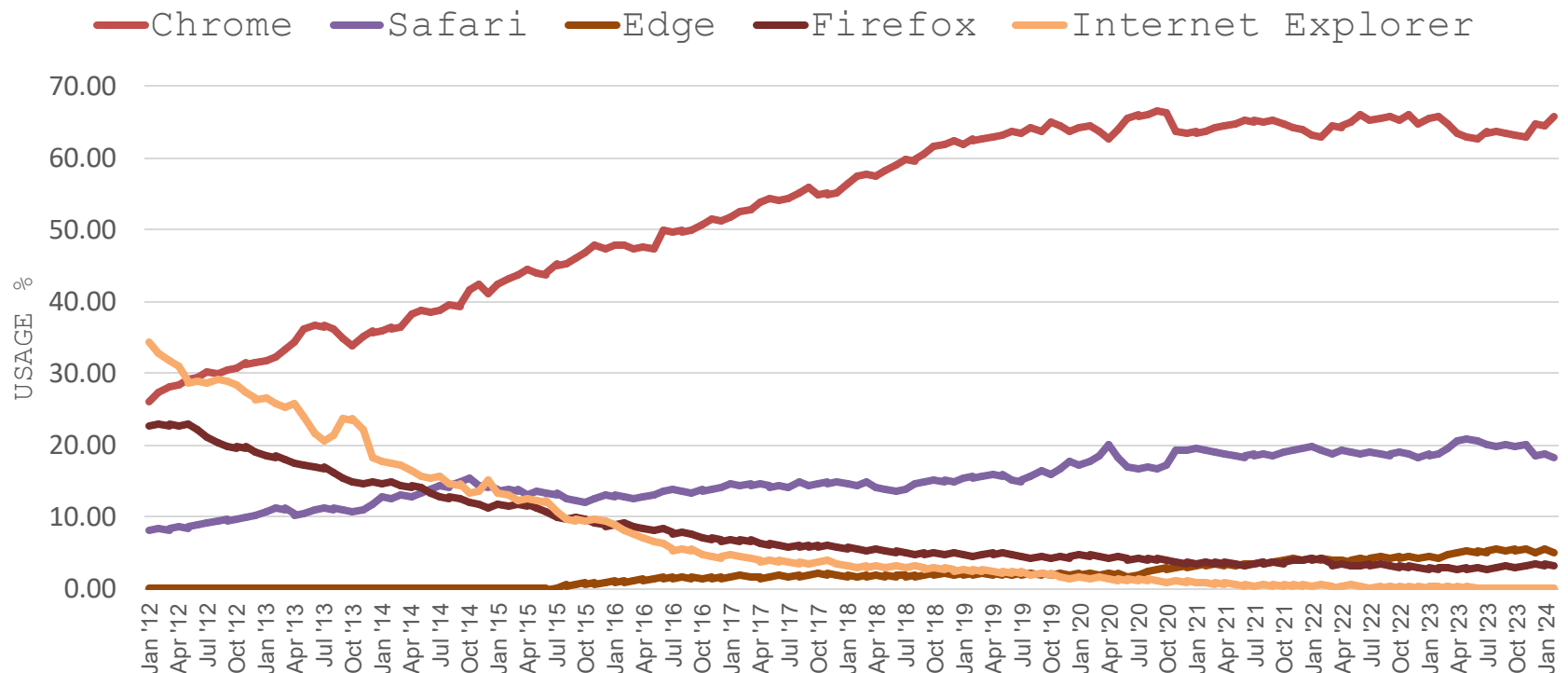
- Worldwide browser market share: now



StatCounter. (2024). *Global market share held by leading internet browsers from January 2012 to February 2024*. Statista. Statista Inc.. Accessed: February 12, 2024. <https://www.statista.com/statistics/268254/market-share-of-internet-browsers-worldwide-since-2009/>

Browsers Evolution

- Worldwide browser market share over time



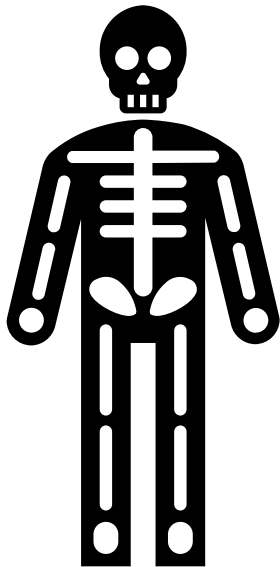
StatCounter. (2024). *Global market share held by leading internet browsers from January 2012 to February 2024*. Statista. Statista Inc.. Accessed: February 12, 2024. <https://www.statista.com/statistics/268254/market-share-of-internet-browsers-worldwide-since-2009/>



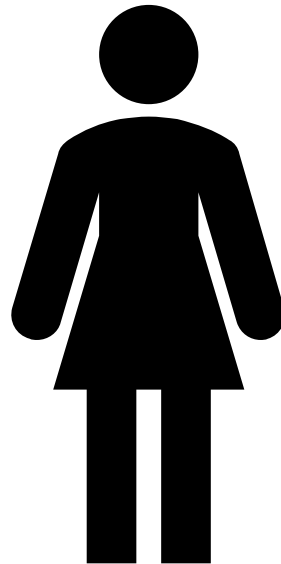
BROWSER COMPONENTS

What is in a page?

**HTML =
content**



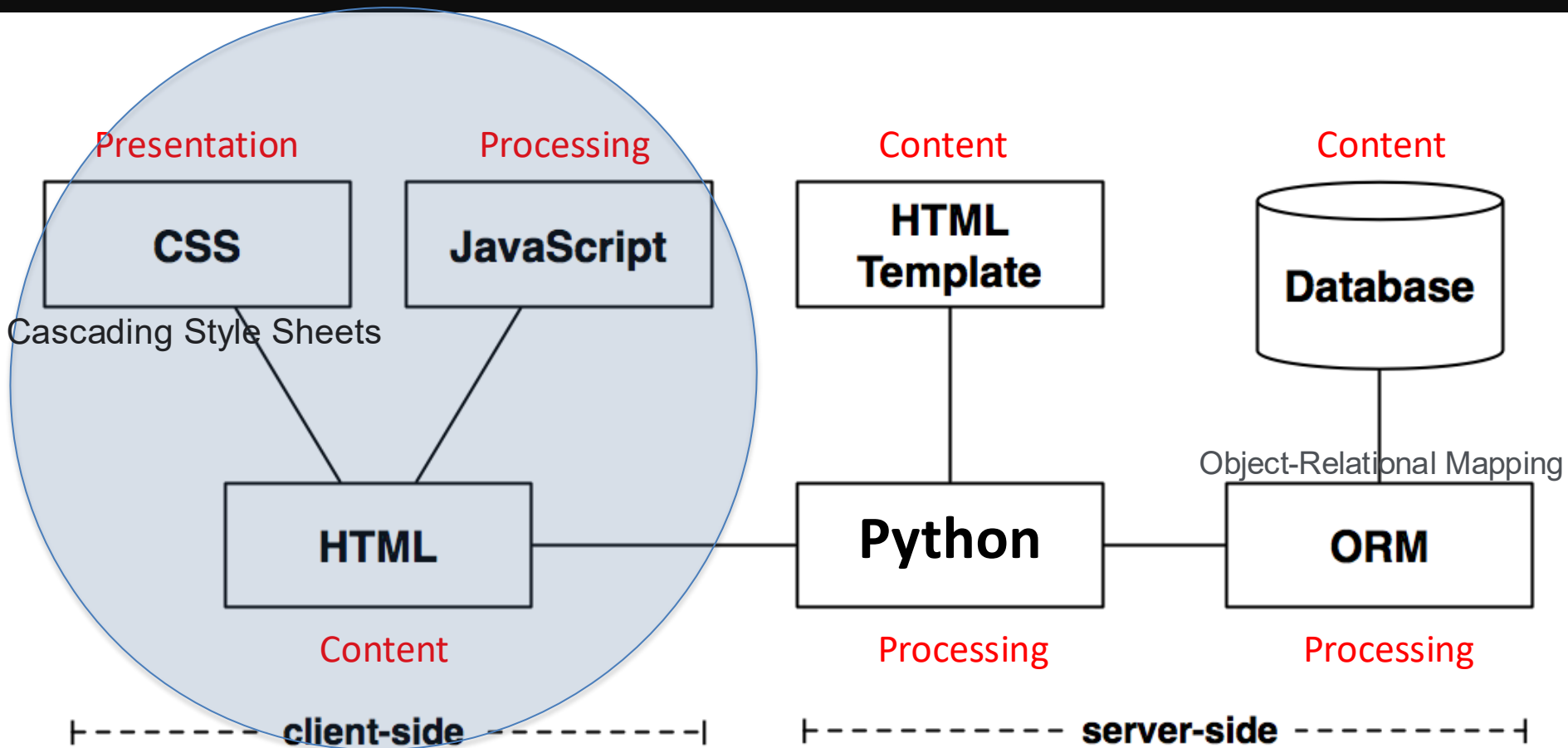
**CSS =
appearance**



**JavaScript =
behavior**



Client-side



The Client-side environment is the Browser

- The browser is the web application runtime
- **Browser components:**
 - **Rendering Engine:** This engine parses HTML and CSS and computes what you see
 - **JavaScript Engine:** This engine executes your JavaScript code
 - **Web APIs (Browser APIs):** This is how Javascript interacts with your page
 - **DOM API:** read/change the page
 - **Events:** clicks, input, submit, keydown...
 - **Fetch:** network requests
 - **Storage:** localStorage/sessionStorage
 - **Timers:** setTimeout/setInterval

Activate Your Web Pages

5th Edition
Includes Ajax and
DOM Scripting



JavaScript

The Definitive Guide

O'REILLY*

David Flanagan

JavaScript
Guide &
Reference

or visit

w3schools

What JavaScript adds to the web

- **HTML + CSS are static**
 - HTML describes *structure*
 - CSS describes *appearance*
 - By themselves, the page doesn't *respond* or *change*
- **JavaScript adds behavior**
 - Respond to **user actions** (click, input, submit, keydown...)
 - Run **logic** (validation, calculations, UI rules)
 - Change the page **without a full reload** (update text, classes, elements)
 - Communicate with servers (**fetch data**) and update the UI

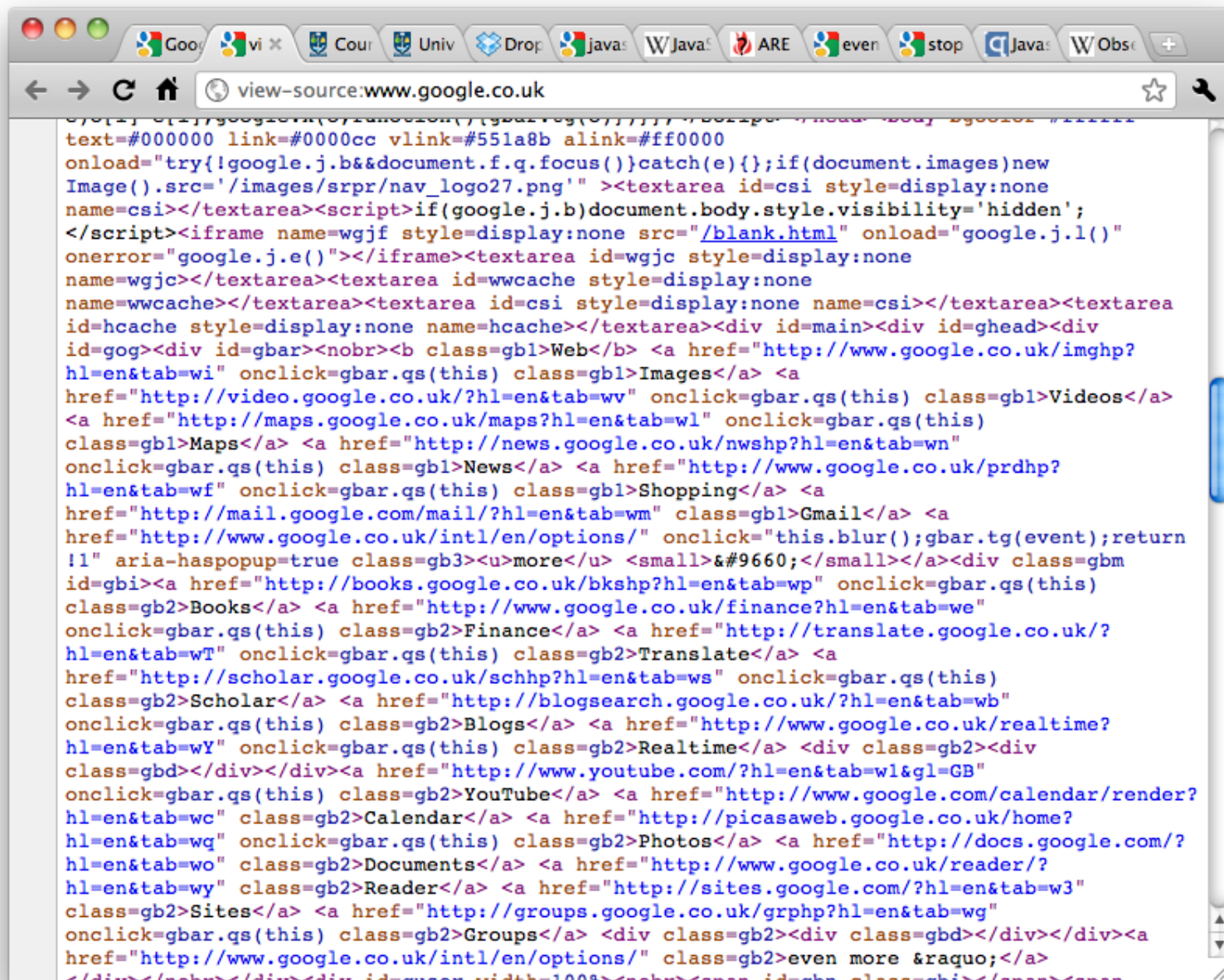
How JavaScript adds behaviour

- JavaScript does not edit the HTML file!
- **JavaScript runs in the browser...**
 - but it needs a way to *read and change* what's on screen
- **Javascript uses DOM = Document Object Model**
 - A **live tree of objects** representing the HTML (elements, attributes, text)
 - JavaScript uses the DOM to select elements, listen for events, update the page



DOCUMENT OBJECT MODEL

HTML Source Code



The image shows a web browser window with the address bar displaying "view-source:www.google.co.uk". The browser's tab bar includes several open tabs: Google, vi, Cour, Univ, Drop, JavaS, WJavaS, ARE, even, stop, JavaS, and WObs. The main content area displays the HTML source code of the Google homepage, which includes various JavaScript functions and HTML elements for navigation and search.

```
text=#000000 link=#0000cc vlink=#551a8b alink=#ff0000
onload="try{!google.j.b&&document.f.q.focus()}catch(e){};if(document.images)new
Image().src='/images/srpr/nav_logo27.png' "><textarea id=csi style=display:none
name=csi></textarea><script>if(google.j.b)document.body.style.visibility='hidden';
</script><iframe name=wgjf style=display:none src="/blank.html" onload="google.j.l()"
onerror="google.j.e()"></iframe><textarea id=wgjc style=display:none
name=wgjc></textarea><textarea id=wwcache style=display:none
name=wwcache></textarea><textarea id=csi style=display:none name=csi></textarea><textarea
id=hcach style=display:none name=hcach></textarea><div id=main><div id=ghead><div
id=gog><div id=gbar><noobr><b class=gbl>Web</b> <a href="http://www.google.co.uk/imghp?
hl=en&tab=wi" onclick=gbar.qs(this) class=gbl>Images</a> <a
href="http://video.google.co.uk/?hl=en&tab=vw" onclick=gbar.qs(this) class=gbl>Videos</a>
<a href="http://maps.google.co.uk/maps?hl=en&tab=w1" onclick=gbar.qs(this)
class=gbl>Maps</a> <a href="http://news.google.co.uk/nwshp?hl=en&tab=wn"
onclick=gbar.qs(this) class=gbl>News</a> <a href="http://www.google.co.uk/prdhp?
hl=en&tab=wf" onclick=gbar.qs(this) class=gbl>Shopping</a> <a
href="http://mail.google.com/mail/?hl=en&tab=wm" class=gbl>Gmail</a> <a
href="http://www.google.co.uk/intl/en/options/" onclick="this.blur();gbar.tg(event);return
!1" aria-haspopup=true class=gb3><u>more</u> <small>&#9660;</small></a><div class=gbm
id=gbi><a href="http://books.google.co.uk/bkshp?hl=en&tab=wp" onclick=gbar.qs(this)
class=gb2>Books</a> <a href="http://www.google.co.uk/finance?hl=en&tab=we"
onclick=gbar.qs(this) class=gb2>Finance</a> <a href="http://translate.google.co.uk/?
hl=en&tab=wT" onclick=gbar.qs(this) class=gb2>Translate</a> <a
href="http://scholar.google.co.uk/schhp?hl=en&tab=ws" onclick=gbar.qs(this)
class=gb2>Scholar</a> <a href="http://blogsearch.google.co.uk/?hl=en&tab=wb"
onclick=gbar.qs(this) class=gb2>Blogs</a> <a href="http://www.google.co.uk/realtime?
hl=en&tab=wY" onclick=gbar.qs(this) class=gb2>Realtime</a> <div class=gb2><div
class=gbd></div></div><a href="http://www.youtube.com/?hl=en&tab=w1&gl=GB"
onclick=gbar.qs(this) class=gb2>YouTube</a> <a href="http://www.google.com/calendar/render?
hl=en&tab=wc" class=gb2>Calendar</a> <a href="http://picasaweb.google.co.uk/home?
hl=en&tab=wq" onclick=gbar.qs(this) class=gb2>Photos</a> <a href="http://docs.google.com/?
hl=en&tab=wo" class=gb2>Documents</a> <a href="http://www.google.co.uk/reader/?
hl=en&tab=wy" class=gb2>Reader</a> <a href="http://sites.google.com/?hl=en&tab=w3"
class=gb2>Sites</a> <a href="http://groups.google.co.uk/grphp?hl=en&tab=wg"
onclick=gbar.qs(this) class=gb2>Groups</a> <div class=gb2><div class=gbd></div></div><a
href="http://www.google.co.uk/intl/en/options/" class=gb2>even more &raquo;</a>
</div></noobr></div><div id=gbox width=100%><noobr><div id=gbr class=gb1></div></div>
```

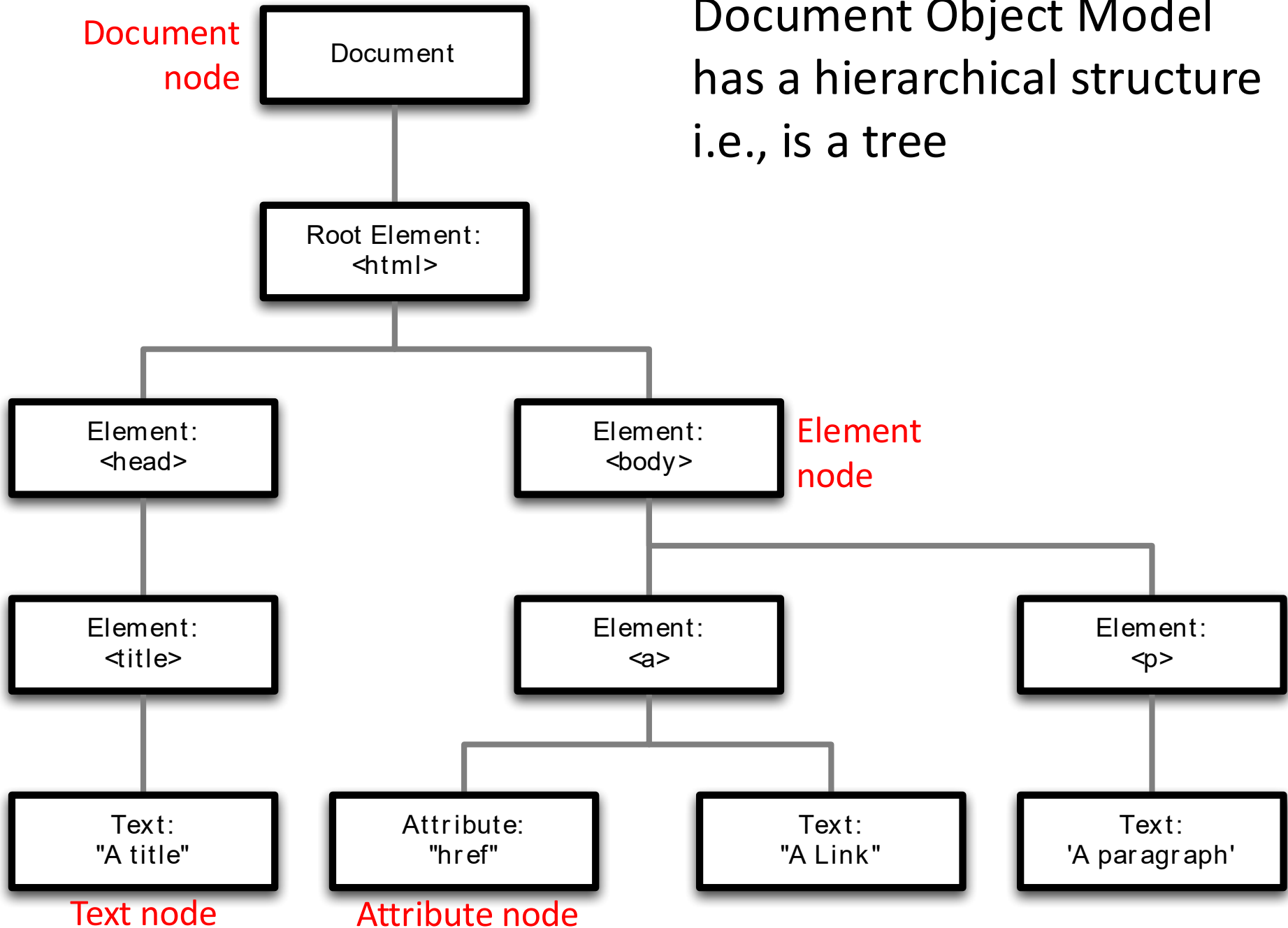
Document Object Model

*“The Document Object Model is a platform- and language-neutral interface that will allow **programs** and scripts to dynamically access and update the content, structure and style of documents.*

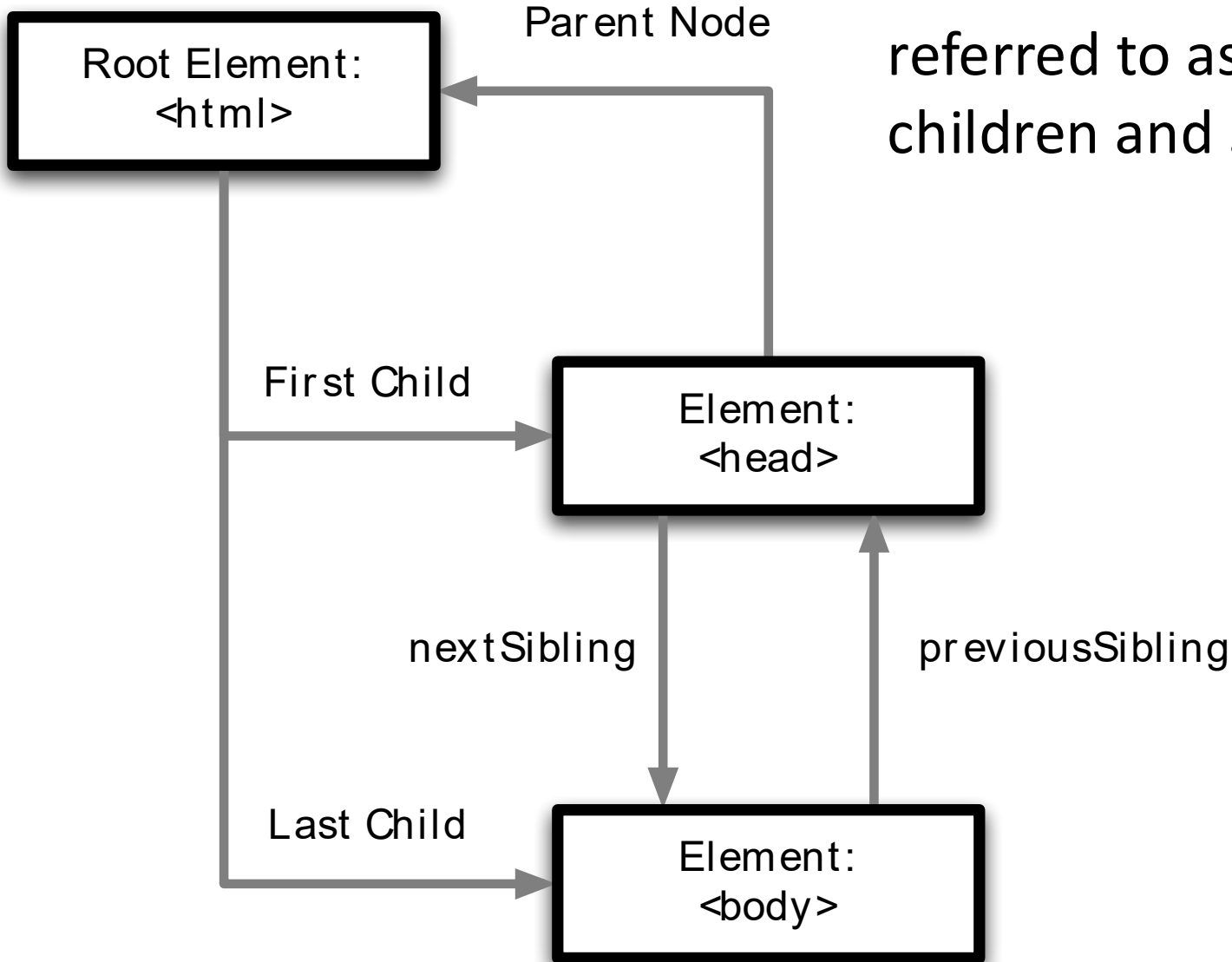
The document can be further processed and the results of that processing can be incorporated back into the presented page.”

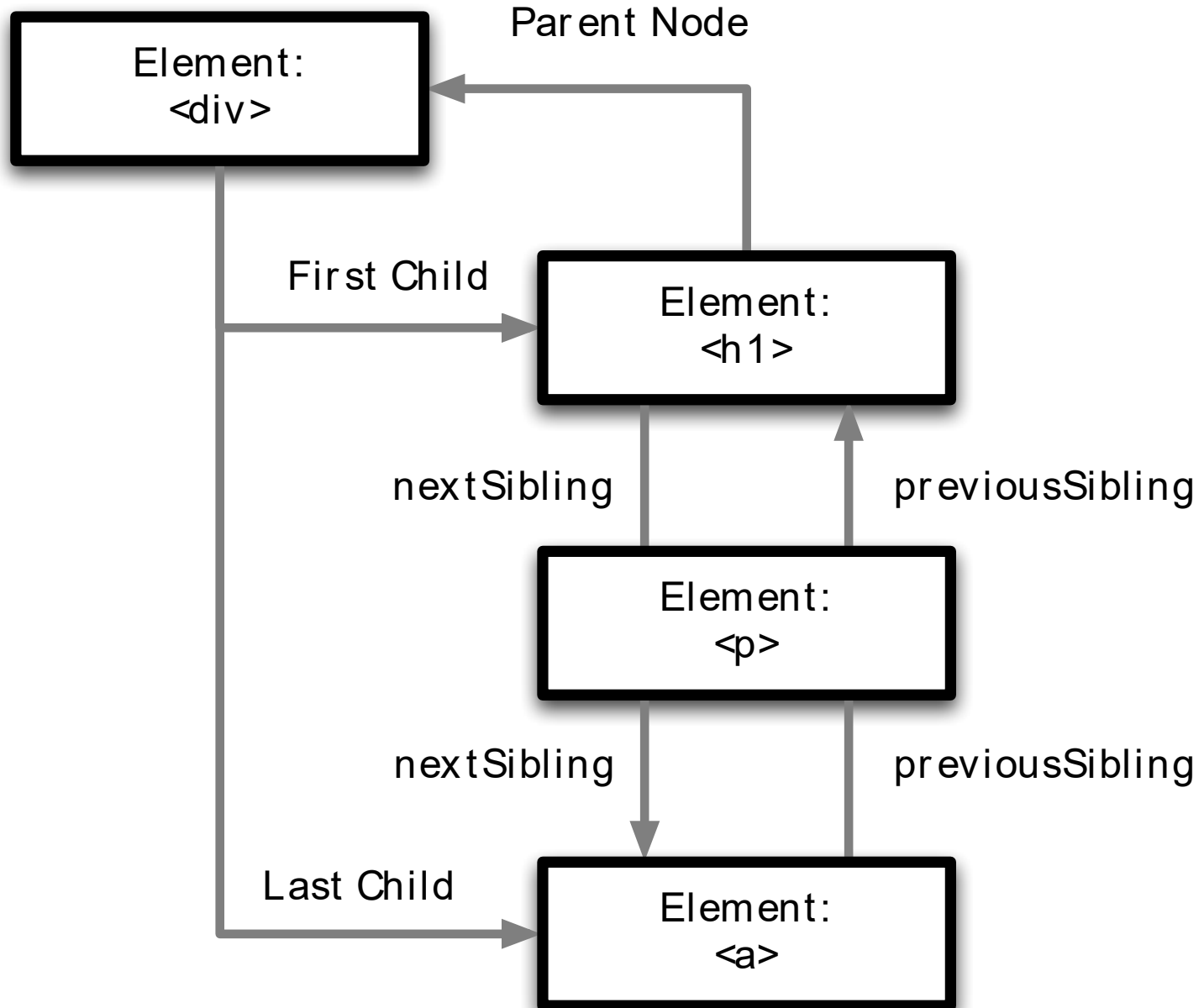
[DOM Standard \(whatwg.org\)](http://whatwg.org)

Document Object Model
has a hierarchical structure
i.e., is a tree



Parts of the tree can be referred to as parents, children and siblings.







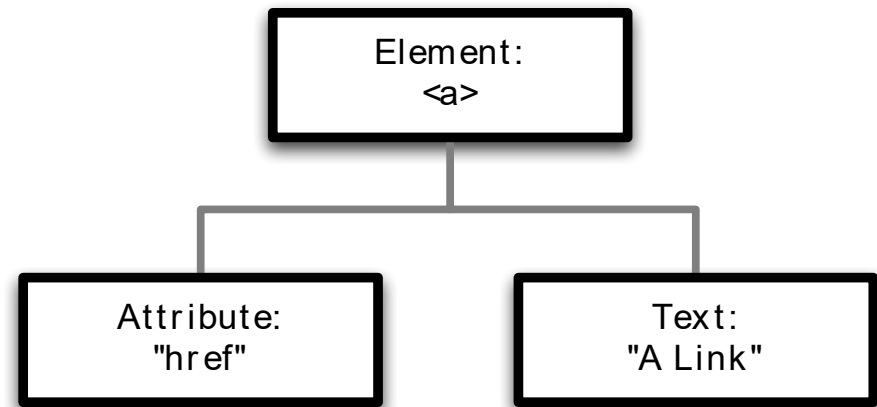
Why do we need DOM in the client side?

- A **web page is a document** that can be either displayed in the browser window or as the HTML source.
- In both cases, it is the same document but the Document Object Model (DOM) representation allows it to be **manipulated**.

Element Properties & Methods

Properties:

`someElement.innerHTML` - the text value
`someElement.nodeName` - the name
`someElement.nodeValue` - the value
`someElement.parentNode` - the parent node
`someElement.childNodes` - the child nodes
`someElement.attributes` - the attributes nodes



Methods:

`someElement.getElementById(id)` - get the element with a specified id
`someElement.getElementsByTagName(name)` - get all elements with a specified tag name
`someElement.appendChild(node)` - insert a child node
`someElement.removeChild(node)` - remove a child node

Finding HTML Elements

When you want to access HTML elements with JavaScript, you have to find the elements first.

There are a couple of ways to do this:

- Finding HTML elements by id:
`document.getElementById(id);`
- Finding HTML elements by tag name:
`document.getElementsByTagName(tag_name);`
- Finding HTML elements by class name

`document.getElementsByClassName(class_name);`

- Finding HTML elements by CSS selectors
`document.querySelectorAll(CSS_selector).`

Finding HTML elements by ID

```
<html>
<body>

<p id="intro">Hello World!</p>

<script type="text/javascript">
  txt=document.getElementById("intro")
    .innerHTML;
  document.write("<p>The
    text from the intro
    paragraph: " + txt + "</p>");
</script>

</body>
</html>
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_getelementbyid

Finding HTML Elements by tag name

```
<html>
<body>

  <h2>JavaScript HTML DOM</h2>
  <p>Finding HTML Elements by Tag Name.</p>
  <p>This example demonstrates the <b>getElementsByTagName</b>
method.</p>
  <p id="demo"></p>

  <script>
    const element = document.getElementsByTagName("p");

    document.getElementById("demo").innerHTML
      = 'The text in first paragraph (index 0)
        is: ' + element[0].innerHTML;
  </script>

</body>
</html>
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_getelementsbytagname2

Finding HTML Elements by class name

```
<html>
<body>

  <h2>JavaScript HTML DOM</h2>
  <p class="intro">Finding HTML Elements by Class Name.</p>
  <p class="intro">This example demonstrates
                        the<b>getElementsByTagName</b> method.</p>
  <p id="demo"></p>

  <script>
    const x = document.getElementsByClassName("intro");
    document.getElementById("demo").innerHTML =
      'The first paragraph (index 0) with class="intro" is: '
      + x[0].innerHTML;
  </script>

</body>
</html>
```

Finding HTML Elements by CSS selector

```
<html>
<body>

  <h2>JavaScript HTML DOM</h2>
  <p class="intro">Finding HTML Elements by Query Selector.</p>
  <p class="intro">This example demonstrates
                    the<b>getElementsByName</b> method.</p>
  <p id="demo"></p>

  <script>
    const x = document.querySelectorAll("p.intro");
    document.getElementById("demo").innerHTML =
      'The first paragraph (index 0) with class="intro" is: '
      + x[0].innerHTML;
  </script>

</body>
</html>
```

DOM Advantages

- **Tree structure makes the DOM easy to traverse**
 - Elements can be accessed one or more times
- **Structure of the Tree is modifiable**
 - Values/Elements/Structure can be added, changed and modified
- **It is a standard of the W3C**
 - i.e. the unofficial/official law of the jungle

DOM Disadvantages

- **Resource Intensive** consuming lots of memory
 - it needs to be fully loaded in main memory
- **Can be slow**
 - Depends on the size and complexity of the Tree
- **May not be the best choice for all devices**
 - i.e. graphics intensive applications or games
- **A better alternative** might be to use the **Canvas**, directly
 - i.e. **OpenGL**

Working with the DOM

Javascript

Jquery

XHTML

CSS

XML

SAX/DOM Parsing

AJAX

MORE INFO...

W3Schools DOM Tutorial:

(https://www.w3schools.com/whatis/whatis_html5dom.asp)

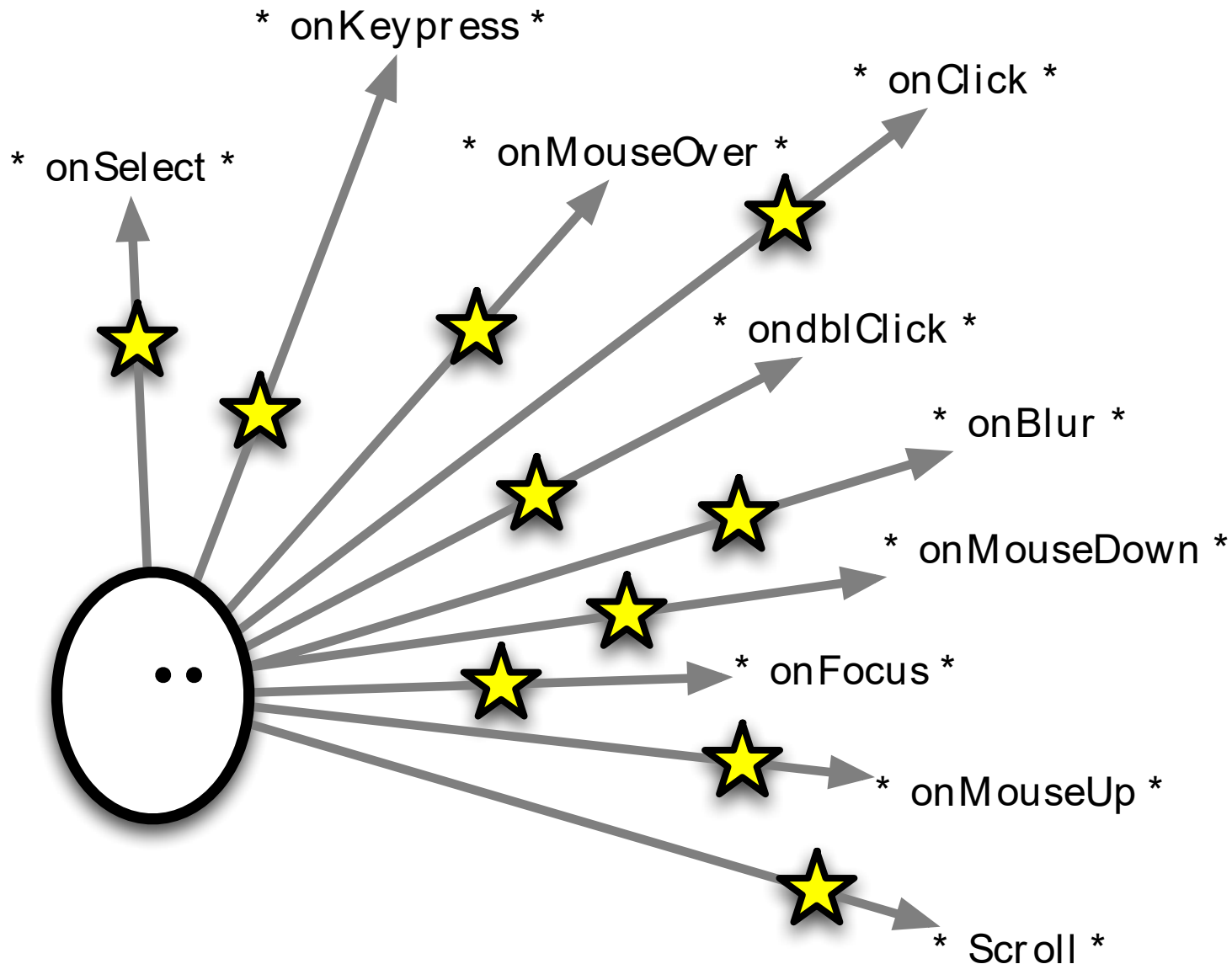
Video: An Inconvenient API: The Theory of the DOM

(<https://www.youtube.com/watch?v=Y2Y0U-2qJMs>)

A security camera is mounted on the upper right corner of the wall, pointing downwards and to the left. It is a white, rectangular device with a black lens and a small antenna.

Tracking User Activity

Handle this...



Event Object

- Each event has an associated object
- Part of the DOM
- The Event Object provides information about:
 - target element in which the event occurred
 - state of the keyboard keys
 - location of the mouse cursor
 - state of the mouse buttons

Event Handling

- Like any interactive application, events can be caught and used to execute functions
- You can add EventListeners to any DOM object
- For example, user input from forms can be validated on the client-side using JavaScript

```
<form name="login_form" action="login" onsubmit="return  
validateForm()" method="post">
```

```
    First name: <input type="text" name="fname">
```

```
    <input type="submit" value="Submit">
```

```
</form>
```

Event Flow

- Each event object has an 'Event Target', e.g., any node in the DOM tree from where an event originated
- There are two main types of event flow, handling events targeted at nested elements
 - event capture (*global handling*)
 - event bubbling (*local handling*)
- Eventflow follows a “**RoundTrip**” pattern/model

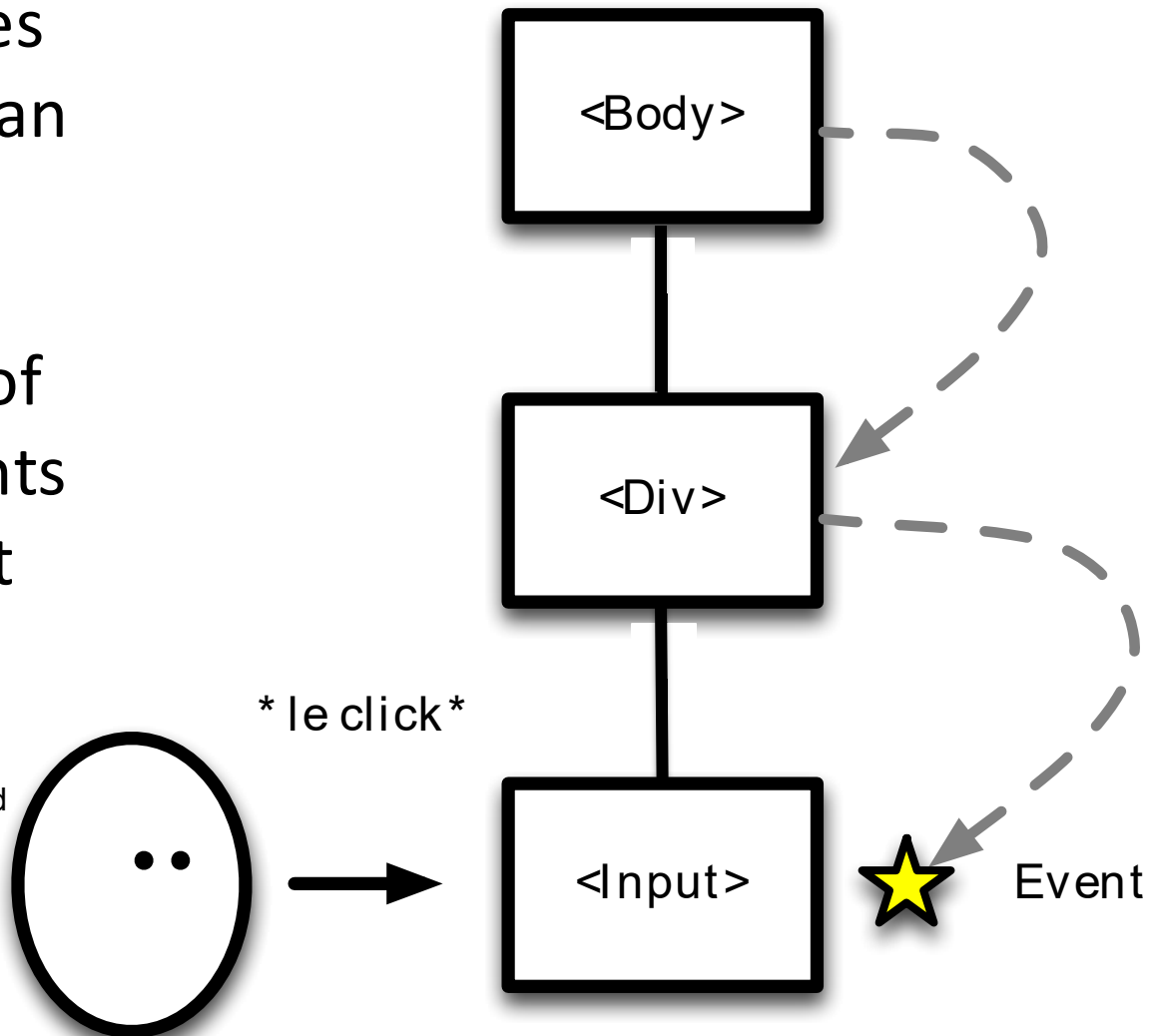
Event Capture

- The event propagates downwards through an element's ancestors
- Any event listeners of the ancestor elements will be executed first

[`addEventListener\(type, listener, useCapture\)`](#)

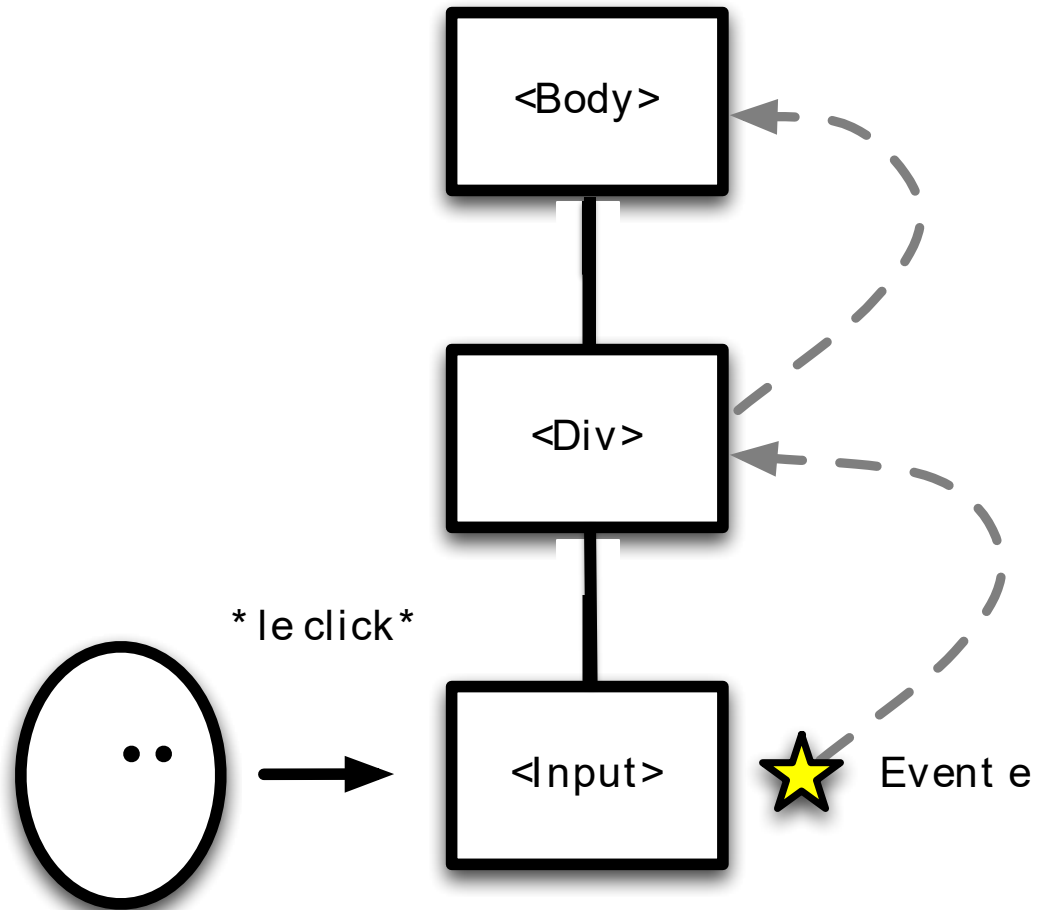
method of the `EventTarget` interface sets up a function that will be called whenever the specified event is delivered to the target.

[`alert\(\)`](#) instructs the browser to display a dialog with an optional message, and to wait until the user dismisses the dialog.



Event Bubbling

- The event propagates upwards through an element's ancestors
- Any event listeners of the element will be executed first
- Ancestors can then potentially handle the event



Remember...

- Event handling order under event capture:
handler1() - > handler2() -> handler3()
- Event handling order under event bubbling:
handler3() - > handler2() -> handler1()

JAVASCRIPT FRAMEWORKS

(TBC)

JavaScript Frameworks

- Ember
 - initially released in December 2011
 - Used less than more modern frameworks but still
 - Benefits: stability, community support, clever coding principles
- Angular
 - open-source web application framework led by the Angular Team at (originally AngularJS)
 - released in 2016
 - a component-based framework which uses declarative HTML templates which are translated to optimised JavaScript instructions.
 - Angular uses TypeScript, a superset of JavaScript

JavaScript Frameworks

- Vue (Pronounce as view)
 - released in 2014
 - Vue, like AngularJS, extends HTML with some of its own code, but mainly relies modern, standard JavaScript
- React
 - Released by Facebook in 2013
 - Technically a library for rendering UI components
 - used in combination with other libraries to make applications i.e. React and React Native enable developers to make mobile applications, React and ReactDOM enable them to make web applications, etc.
 - React extends JavaScript with HTML-like syntax, known as JSX

Summary

- Clients change on a regular basis – Chrome has taken over from IE
- Client-side environment relies on HTML, CSS and JavaScript as basic building blocks
- There are JS Frameworks that can help to build web applications
- Important to understand DOM and event handling